



Web Architecture and Version Control

Web Design And Programming

M.Sc. Tran Quang Toan

Faculty of Data Science and Artificial Intelligence



Week 1 Outline

1. How The Web Works

2. Version Control With Git And GitHub

3. DS & AI Perspective

4. Lab & MCQ Review



Learning Objectives

By the end of this session, you will be able to:

1. Explain how the web works using the client-server model.
2. Describe the HTTP request/response lifecycle and common status codes.
3. Parse a URL into protocol, domain, and path.
4. Use core Git workflow: init, add, commit, branch, push, pull.
5. Distinguish branch, fork, and pull request on GitHub.



How The Web Works



Internet vs. Web

- **Internet:** A global network infrastructure connecting computers.
- **Web (WWW):** A service running on top of the Internet using HTTP/HTTPS.
- The web is one part of the Internet, alongside email, SSH, FTP, and others.
- A browser is a client that requests and renders web resources.



Client-Side and Server-Side

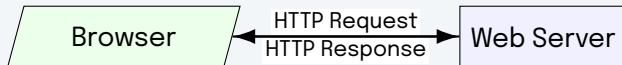
Client-Side (Frontend)

- Runs in the browser
- HTML for structure
- CSS for presentation
- JavaScript for behavior

Server-Side (Backend)

- Receives requests
- Executes application logic
- Interacts with database
- Sends response to client

HTTP Request/Response Lifecycle



- Client sends method + URL + headers (+ optional body).
- Server processes and returns status code + headers + content.



HTTP Methods and Status Codes

Common Methods

- GET: Retrieve resource
- POST: Create/send data
- PUT/PATCH: Update resource
- DELETE: Remove resource

Exam-Focused Codes

- 200 OK: Success
- 301/302: Redirect
- 404 Not Found: Missing resource
- 500 Internal Server Error: Server failed

URL Structure

Example URL:

`https://example.com/courses/week01/index.html`

- **Protocol:** `https`
- **Domain name:** `example.com`
- **Path:** `/courses/week01/index.html`
- **Absolute path:** starts from root `/`
- **Relative path:** resolved from current page location



Page Anatomy (From Textbook Ch. 2)

- A web page is a document built from HTML elements.
- CSS controls visual style and layout.
- JavaScript can manipulate the DOM and react to events.
- The browser fetches linked resources: CSS, JS, images, fonts.
- Performance depends on request count, payload size, and caching.



Version Control With Git And GitHub



Why Version Control Matters

- Keeps full history of project changes.
- Supports safe teamwork and parallel development.
- Allows rollback when bugs are introduced.
- Documents technical decisions through commit messages.
- Essential in web projects, data science pipelines, and AI experiments.



Git Core Concepts

- **Repository:** Project history database.
- **Working directory:** Files currently edited.
- **Staging area (index):** Curated set of changes for next commit.
- **Commit:** Snapshot with author, timestamp, and message.
- **Branch:** Independent line of development.



Basic Git Workflow

```
1 git init
2 git status
3 git add index.html
4 git commit -m "Create initial webpage"
5 git branch feature/navbar
6 git switch feature/navbar
```

- Commit early, commit often, with meaningful messages.

Working with Remote (GitHub)

```
1 git remote add origin <repository -url>
2 git push -u origin main
3 git pull origin main
```

- **Push:** Upload local commits to remote.
- **Pull:** Fetch + integrate latest remote commits.
- Pull before push to reduce merge conflicts.



Branch vs. Fork vs. Pull Request

- **Branch:** Parallel work inside the same repository.
- **Fork:** Your personal copy of another repository.
- **Pull Request (PR):** Request to merge proposed changes.

Typical open-source flow:

1. Fork repository.
2. Create feature branch.
3. Commit and push changes.
4. Open PR for review and merge.



DS & AI Perspective



Why This Matters for Data Science & AI

- Web knowledge is required to deploy AI services for users.
- HTTP is the foundation for API inference endpoints.
- Git tracks model code, feature engineering scripts, and experiments.
- .gitignore prevents committing heavy folders (e.g., datasets, `node_modules`).
- Team collaboration quality depends on clean commits and PR reviews.



Bridge to Future Weeks

- Today: Browser-server communication and source control.
- Next: Semantic HTML structure and accessible content.
- Later: CSS layout, JavaScript DOM, Node.js backend, APIs.
- End goal: Build complete web applications connected to data/AI workflows.



Lab & MCQ Review



In-Class Lab Tasks

1. Install Git and a code editor.
2. Create a project folder and run `git init`.
3. Create `index.html` and write a basic HTML page.
4. Add a `.gitignore` file.
5. Run `git add .` and `git commit -m "Week 1 lab"`.
6. Create a GitHub repository and push your code.



MCQ Drill (Sample Questions)

1. Which status code means success? **(200)**
2. Which command moves changes to staging? **(git add)**
3. Which action gets updates from remote? **(git pull)**
4. Branch and Fork differ mainly by: **scope of repository ownership**
5. URL component `https` is: **protocol**



Summary & Q&A

- The web follows a client-server architecture over HTTP.
- URL, request methods, and status codes are core exam topics.
- Git enables history, rollback, collaboration, and safer development.
- GitHub workflows use branch/fork/PR for code review and integration.

Questions & Answers